

Guía Paso a Paso: Mapeo, Migración y Modificación de Modelos en Sequelize

Esta guía detalla el funcionamiento del mapeo objeto-relacional (ORM) en Sequelize, analiza detalladamente el código del proyecto y explica cómo agregar una nueva columna a una tabla existente mediante migraciones.

1. Conceptos Fundamentales: Modelos, Tablas y Migraciones

En el desarrollo backend con Node.js, Sequelize actúa como intermediario (ORM) entre el código de la aplicación y la base de datos PostgreSQL.

- **La Base de Datos (PostgreSQL):** Es el almacenamiento de datos físico en disco. Consta de tablas, columnas reales, claves primarias y foráneas.
- **El Modelo (Código JavaScript):** Representa la estructura de datos que tu aplicación maneja en memoria. Sequelize mapea cada atributo del modelo a una columna física.
- **La Migración (Control de Versiones):** Script ordenado que ejecuta operaciones DDL en la base de datos (crear tablas, modificar columnas, agregar índices) para asegurar que la base de datos esté en el mismo estado en cualquier entorno.

IMPORTANTE:

Para agregar o modificar cualquier columna, es obligatorio actualizar el modelo en JavaScript Y ejecutar una migración en la base de datos. De lo contrario, ocurrirán fallas graves por falta de sincronización.

2. Comentarios Detallados del Código Existente

A continuación, se detalla el funcionamiento de los archivos del proyecto utilizando comentarios detallados línea por línea.

A. Modelo de Bebida (src/models/bebida.js)

```
'use strict';
export default (sequelize, DataTypes) => {
  // Define el modelo 'Bebida'. Sequelize asociará esto automáticamente
  // con la tabla física 'bebidas' indicada en las opciones.
  const Bebida = sequelize.define('Bebida', {

    // Atributo: nombre de la bebida
    nombre: {
      type: DataTypes.STRING(200), // Mapea a VARCHAR(200) en PostgreSQL
      allowNull: false, // Aplica la restricción NOT NULL
      unique: true, // Crea un índice único (no nombres repetidos)
    },

    // Atributo: precio de la bebida
    precio: {
      type: DataTypes.DECIMAL(15, 2), // Mapea a NUMERIC(15,2) (perfecto para dinero)
      allowNull: false, // Restricción NOT NULL
      defaultValue: 0, // Si no se asigna al crear, por defecto vale 0
    },

    // COMENTADO: Este campo representa la columna de stock.
    // Aunque exista la columna en PostgreSQL físicamente, al estar comentada aquí,
    // Sequelize NO la incluirá en sus selects ni updates en memoria.
    // stock: {
    //   type: DataTypes.INTEGER,
    //   allowNull: false,
    //   defaultValue: 0,
    // },

    // Atributo: descripción detallada de la bebida
    descripcion: {
      type: DataTypes.TEXT, // Mapea a TEXT en PostgreSQL (longitud ilimitada)
      allowNull: true, // Permite valores nulos (campo opcional)
    },
  }, {
    tableName: 'bebidas', // Nombre real de la tabla física en la base de datos
    timestamps: false, // Desactiva la creación automática de 'createdAt' y 'updatedAt'

    // Hooks: Funciones que se ejecutan automáticamente en el ciclo de vida del modelo
    hooks: {
      // Antes de guardar un registro (crear o actualizar), transforma el nombre a mayúsculas
      beforeSave: (instance) => {
        if (instance.nombre) instance.nombre = instance.nombre.toUpperCase();
      },
    },
  });

  return Bebida;
};
```

B. Inicializador de Modelos (src/models/index.js)

Este archivo se encarga de levantar la conexión a la base de datos basándose en el entorno activo (development, test, production), inicializar los modelos pasándoles la conexión y configurar las relaciones de clave foránea.

```
// src/models/index.js
import { Sequelize, DataTypes } from 'sequelize';
import path from 'path';
import { fileURLToPath } from 'url';
import dbConfig from '../config/database.cjs';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const env = process.env.NODE_ENV || 'development';
const config = dbConfig[env]; // Obtiene la configuración de conexión

// Inicializa la conexión física a la Base de Datos
const sequelize = new Sequelize(
  config.database,
  config.username,
  config.password,
  config
);

import BarraModel from './barra.js';
import BebidaModel from './bebida.js';
import VentaModel from './venta.js';
import DetalleVentaModel from './detalleventa.js';

// Inicializa cada modelo individualmente
const Barra = BarraModel(sequelize, DataTypes);
const Bebida = BebidaModel(sequelize, DataTypes);
const Venta = VentaModel(sequelize, DataTypes);
const DetalleVenta = DetalleVentaModel(sequelize, DataTypes);

// CONFIGURACIÓN DE RELACIONES (Foreign Keys)
// 1. Relación Ventas <-> DetalleVenta (Una venta tiene muchos detalles)
Venta.hasMany(DetalleVenta, { foreignKey: 'venta_id', as: 'Detalle' });
DetalleVenta.belongsTo(Venta, { foreignKey: 'venta_id', as: 'Venta' });

// 2. Relación Bebidas <-> DetalleVenta (Una bebida tiene muchos detalles de venta)
Bebida.hasMany(DetalleVenta, { foreignKey: 'bebida_id', as: 'Detalle' });
DetalleVenta.belongsTo(Bebida, { foreignKey: 'bebida_id', as: 'Bebida' });

// 3. Relación Barras <-> DetalleVenta (Una barra tiene muchos detalles de venta)
Barra.hasMany(DetalleVenta, { foreignKey: 'barra_id', as: 'Detalle' });
DetalleVenta.belongsTo(Barra, { foreignKey: 'barra_id', as: 'Barra' });

export {
  sequelize,
  Sequelize,
  Barra,
  Bebida,
  Venta,
  DetalleVenta,
};
```

3. Guía de Ejecución: Cómo agregar una nueva columna

Supongamos que deseas activar el control de **stock** en tus bebidas. Para agregarlo de manera profesional a tu base de datos y a tu modelo de Sequelize, debes seguir detalladamente el flujo estructurado en 5 pasos:

Paso 1: Generar una nueva migración desde la consola

En entornos de producción, **nunca debes editar un archivo de migración antiguo** que ya haya sido ejecutado en la base de datos. Modificar archivos antiguos corrompe el historial de control de versiones y generará problemas al desplegar. En su lugar, debes generar un nuevo archivo de migración vacío ejecutando el siguiente comando en la consola (desde el directorio *app* del proyecto):

```
npx sequelize-cli migration:generate --name agregar-stock-bebidas
```

Esto generará un archivo vacío dentro de la carpeta *app/src/migrations/* con un nombre con prefijo temporal (por ejemplo: 20260616180155-agregar-stock-bebidas.js).

Paso 2: Programar la migración (up y down)

Abre el archivo recién creado. Debes definir dos funciones asíncronas: up (qué cambios aplicar en la BD) y down (cómo revertir esos mismos cambios en caso de rollback). Edita el archivo para estructurarlo así:

```
'use strict';

module.exports = {
  // up: Define qué cambios se aplicarán en la Base de Datos
  async up(queryInterface, Sequelize) {
    // Agrega una columna a la tabla 'bebidas' llamada 'stock'
    await queryInterface.addColumn('bebidas', 'stock', {
      type: Sequelize.INTEGER, // Tipo de dato entero
      allowNull: false, // Restricción NOT NULL
      defaultValue: 0, // Valor por defecto en los registros existentes
    });
  },

  // down: Define cómo revertir los cambios exactos aplicados en el método 'up'
  async down(queryInterface, Sequelize) {
    // Elimina la columna 'stock' de la tabla 'bebidas'
    await queryInterface.removeColumn('bebidas', 'stock');
  },
};
```

NOTA TÉCNICA:

El uso de 'defaultValue: 0' es indispensable cuando agregamos una columna 'NOT NULL' a una tabla que ya tiene filas existentes. PostgreSQL requiere un valor por defecto para rellenar esos registros previos y cumplir con la restricción.

Paso 3: Aplicar la migración a la base de datos

Para aplicar físicamente la columna nueva en PostgreSQL dentro del contenedor Docker o base de datos local, ejecuta el siguiente comando en la consola de tu proyecto:

```
npx sequelize-cli db:migrate
```

Sequelize comparará tu carpeta de migraciones con la tabla especial SequelizeMeta en tu base de datos y ejecutará la función up() únicamente de los scripts nuevos.

Paso 4: Realizar el mapeo de la columna en el modelo Sequelize

Una vez que la columna física ya existe en la base de datos, debemos actualizar el archivo de definición del modelo (app/src/models/bebida.js) para indicarle al ORM que puede utilizarla. Simplemente descomentamos o agregamos las líneas correspondientes dentro del esquema del modelo:

```
const Bebida = sequelize.define('Bebida', {
  nombre: { ... },
  precio: { ... },

  // Descomentar o añadir la columna en el mapeo:
  stock: {
    type: DataTypes.INTEGER,
    allowNull: false,
    defaultValue: 0,
  },

  descripcion: { ... }
```

Paso 5: ¿Cómo revertir la migración si hay problemas? (Rollback)

Si notas que cometiste algún error en la definición de la columna o el tipo de dato inmediatamente después de aplicar la migración, puedes deshacer el cambio ejecutando:

```
npx sequelize-cli db:migrate:undo
```

Este comando lee el historial de base de datos, identifica el último archivo de migración aplicado y ejecuta su método down() (eliminando la columna stock de forma limpia), borrando el archivo del historial en SequelizeMeta.

4. Tabla de Equivalencias de Tipos de Datos

A continuación, se presenta un resumen con las equivalencias entre los tipos de datos lógicos definidos en los modelos de Sequelize y las columnas físicas correspondientes en PostgreSQL:

Tipo en Sequelize (DataTypes.*)	Tipo en PostgreSQL	Ejemplo de Uso Práctico
INTEGER	INTEGER	Cantidades enteras (stock, cantidad)
DECIMAL(p, s)	NUMERIC(p, s)	Dinero / Importes decimales precisos
STRING(n)	VARCHAR(n)	Textos cortos indexados (nombre, sector)
TEXT	TEXT	Descripciones de longitud ilimitada
BOOLEAN	BOOLEAN	Estados lógicos (activo, disponible)
DATE	TIMESTAMP WITH TIME ZONE	Marcas temporales con zona horaria
DATEONLY	DATE	Fechas puras sin horas (fecha de nacimiento)

5. Buenas Prácticas al Trabajar con Modelos y Migraciones

- Inmutabilidad del esquema:** Nunca edites un archivo de migración viejo que ya haya sido desplegado. Siempre genera uno nuevo.
- Consistencia en las restricciones:** Asegúrate de que las restricciones definidas en la base de datos (por ejemplo, allowNull o defaultValue) coincidan exactamente con las declaradas en tu modelo JS de Sequelize.
- Siempre proveer un método down():** Toda operación destructiva o constructiva debe poder revertirse. Si no lo haces, será imposible realizar rollbacks en producción en caso de despliegues fallidos.
- Manejar transacciones para migraciones complejas:** Para migraciones de alta complejidad con múltiples consultas, agrúpalas dentro de una transacción para evitar que el esquema de la base de datos quede en un estado corrupto intermedio si una de las sentencias falla.